COMPUTER SCIENCE

Algorithm Notes

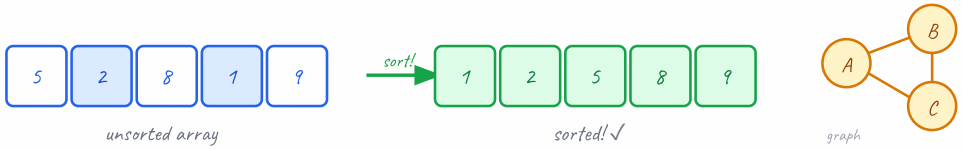
Complete Study Guide

Sorting Algorithms

Greedy Algorithms

Graph Algorithms

Definitions · Numericals · Code · Tips



📋 Table of Contents

Part I — Sorting Algorithms

01. Bubble Sort3

02. Selection Sort4

03. Insertion Sort5

04. Merge Sort6

05. Quick Sort8

06. Heap Sort10

📊 Sorting Complexity Comparison12

Part II — Greedy Algorithms

07. Activity Selection Problem13

08. Fractional Knapsack14

09. Huffman Coding15

10. Job Sequencing with Deadlines17

11. Coin Change (Greedy)18

Part III — Graph Algorithms

12. BFS — Breadth First Search19

13. DFS — Depth First Search20

14. Dijkstra's Algorithm21

15. Bellman-Ford Algorithm23

16. Kruskal's MST25

17. Prim's MST26

18. Floyd-Warshall27

19. Topological Sort28

📊 Graph Algorithms Summary29

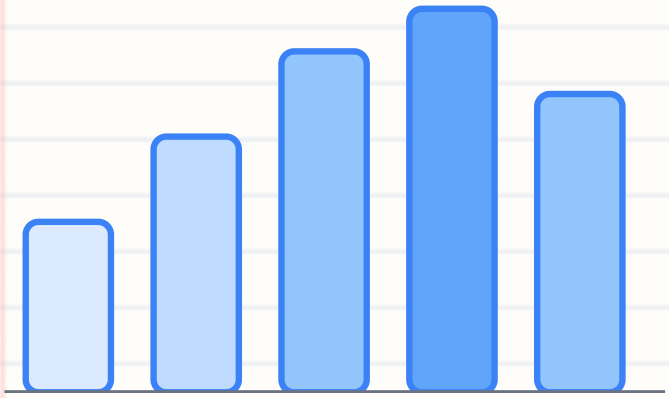
💡 Tip: Each algo has Definition → Complexity → Numerical → Code → Key Points

Part 1

Sorting

Algorithms

"Arrange data in a specific order"



01

Bubble Sort

Easy

O(n²)

📖 Definition

Repeatedly compares **adjacent elements** and swaps them if out of order. Largest element *"bubbles up"* to end in each pass. Like sorting by repeatedly picking bubble to top!

📐 Key Facts

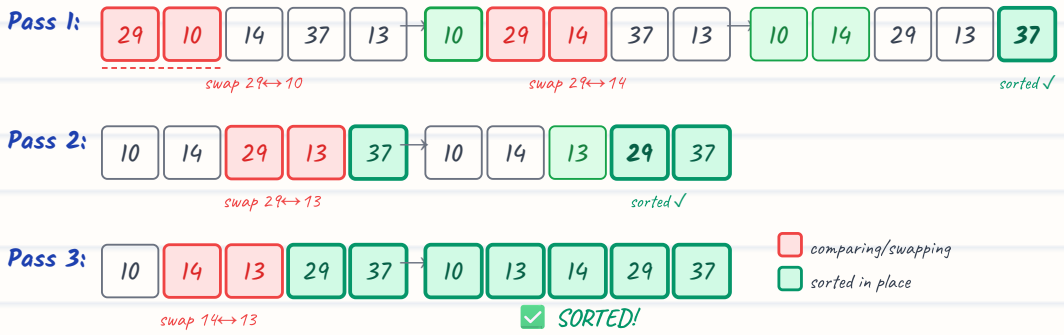
- Passes needed = **n − 1**
- Comparisons in pass i = **n − i**
- Max swaps = n(n−1)/2
- Optimization: stop if no swap in a pass

⚡ Complexity

Case	Time	Space
Best	O(n)	
Average	O(n²)	O(1)
Worst	O(n²)	

✓ Stable ✓ In-Place

🔢 Numerical — Sort [29, 10, 14, 37, 13]



C++ Code

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        bool swapped = false;
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                swap(arr[j], arr[j+1]);
                swapped = true;
            }
        }
        if (!swapped) break; // Already sorted!
    }
}
```

💡 Key Points to Remember

- Best case O(n) when array already sorted (with flag check)
- Called "Bubble" — larger elements bubble to the top each pass
- Total passes = n-1, comparisons in pass i = n-i-1
- Rarely used in practice — only for educational purposes

02

Selection Sort

Easy

O(n²)

📖 Definition

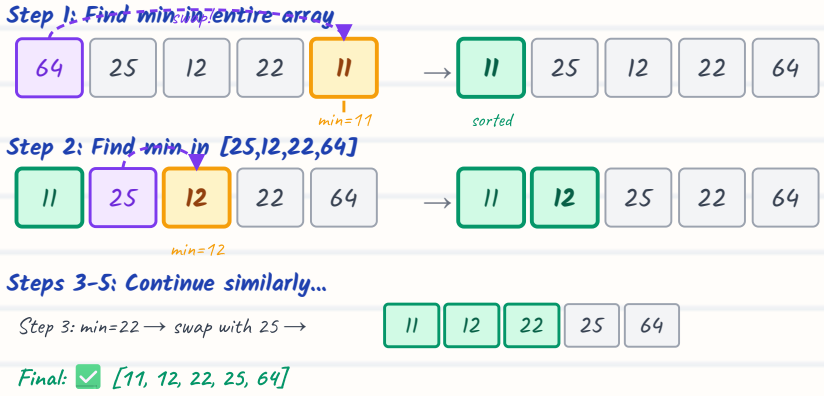
Divides array into **sorted** and **unsorted** parts. Finds minimum from unsorted part and places it at the beginning of unsorted. Repeats until fully sorted.

⚡ Complexity

Case	Time
Best/Avg/Worst	O(n²)
Space	O(1)

✗ Unstable ✓ In-Place

🔢 Numerical — Sort [64, 25, 12, 22, 11]



C++ Code

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++)
            if (arr[j] < arr[minIdx]) minIdx = j;
        swap(arr[minIdx], arr[i]);  // Always n-1 swaps
```

```
        }
    }
```

💡 Key Points

- Always makes exactly **n-1 swaps** — great when write operations are costly
- Unstable: equal elements may change relative order
- Total comparisons = n(n-1)/2 regardless of input

04

Merge Sort

Medium

O(n log n)

📖 Definition

Divide and Conquer: Recursively split array in halves until single elements. Then **merge** sorted halves back. Guaranteed O(n log n) always.

📐 Recurrence

T(n) = 2T(n/2) + O(n)

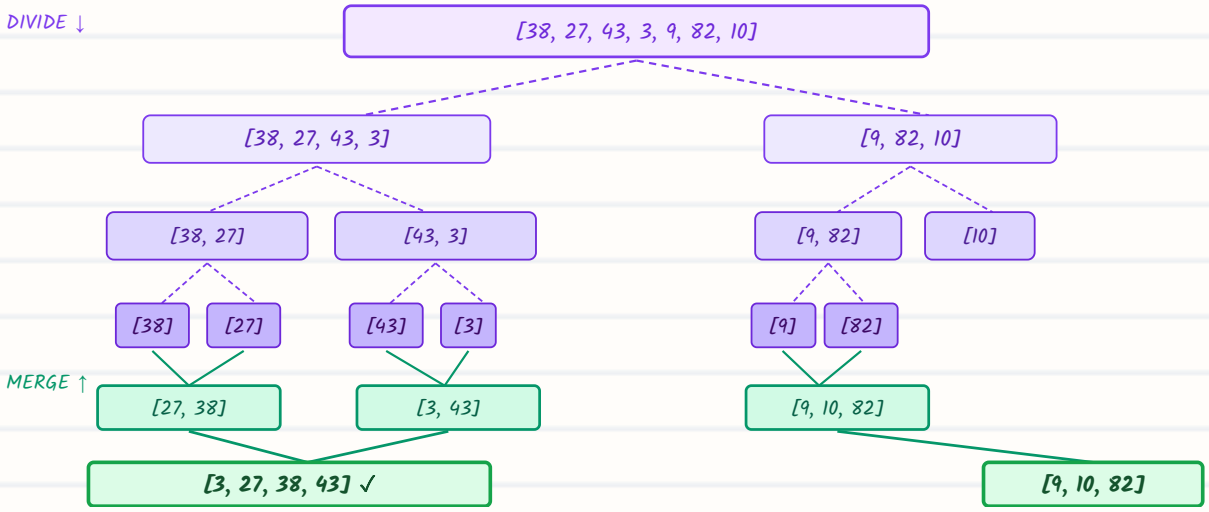
→ By Master Theorem: **O(n log n)**

⚡ Complexity

Case	Time	Space
Best	O(n log n)	
Average	O(n log n)	O(n)
Worst	O(n log n)	

✓ Stable ✗ Extra Space

🔢 Numerical — Sort [38, 27, 43, 3, 9, 82, 10]



C++ Code

```
void merge(int arr[], int l, int m, int r) {
    vector<int> L(arr+l, arr+m+1), R(arr+m+1, arr+r+1);
    int i=0, j=0, k=l;
    while(i<L.size() && j<R.size())
        arr[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
    while(i<L.size()) arr[k++]=L[i++];
    while(j<R.size()) arr[k++]=R[j++];
}
void mergeSort(int arr[], int l, int r) {
    if(l < r) {
        int m = l + (r-l)/2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}
```

💡 Key Points

- Best for **linked lists** — no random access needed for merge
- Used in **external sorting** (when data doesn't fit in RAM)
- Basis of **Timsort** (Python/Java's built-in sort)
- Merge step: compare heads, take smaller — O(n) per level

05

Quick Sort

Medium

O(n log n) avg

📖 Definition

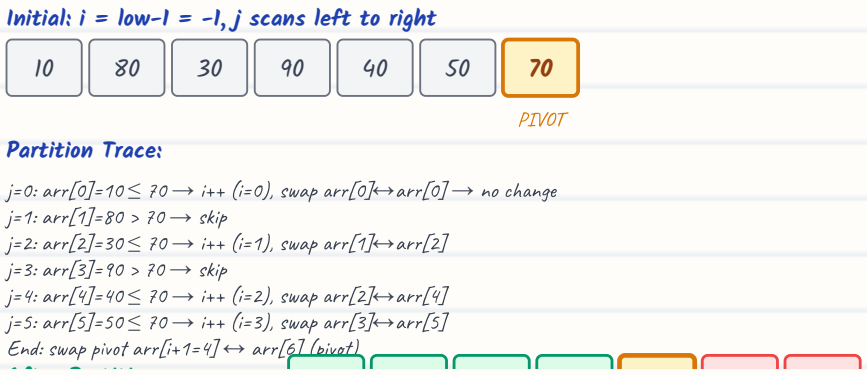
Pick a **pivot** → Partition array so that elements < pivot are left, elements > pivot are right → Recursively sort both halves. Pivot ends up at its correct final position.

⚡ Complexity

Case	Time
Best/Avg	O(n log n)
Worst	O(n²)
Space	O(log n)

✗ Unstable ✓ In-Place ✓ Cache-friendly

🔢 Numerical — Partition step on [10, 80, 30, 90, 40, 50, 70], pivot = 70



C++ Code

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] <= pivot) swap(arr[++i], arr[j]);
    swap(arr[i+1], arr[high]);
    return i + 1;  // pivot's final index
}
```

```
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi-1);   // left of pivot
        quickSort(arr, pi+1, high);  // right of pivot
    }
}
```

💡 Key Points

- Worst case $O(n^2)$ when pivot is always min/max — use random pivot to avoid
- Fastest in practice due to **cache locality**
- C++ `std::sort` uses Introsort = Quick + Heap + Insertion

14

Dijkstra's Algorithm

Medium

O((V+E) log V)

📖 Definition

Single Source Shortest Path for graphs with *non-negative* weights. Uses a **Min-Heap (priority queue)**. Greedily selects nearest unvisited vertex and relaxes its edges.

📐 Relaxation

if dist[u] + w(u,v) < dist[v]:

dist[v] = dist[u] + w(u,v)

⚠️ Fails with negative edges!

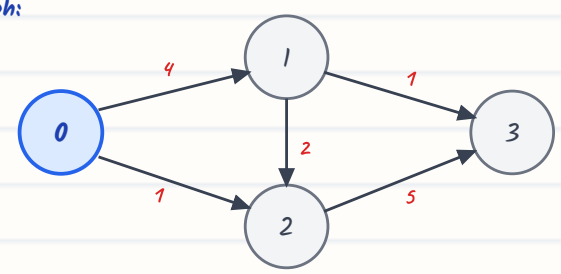
⚡ Complexity

Implementation	Time
Array	$O(V^2)$
Binary Heap	$O((V+E)\log V)$
Space	O(V)

✓ Greedy ✗ No neg edges

🔢 Numerical — Shortest path from node 0

Graph:



Step-by-step dist[] table:

Step	dist[0]	dist[1]	dist[2]	dist[3]
Init	0	∞	∞	∞
Pick 0	0	4	1	relax ∞
Pick 2	0	~~3~~ 1+2=3!	1	6
Pick 1	0	3	1	~~6~~ 3+1=4!
Pick 3	0	3	1	4
Final dist:	**0**	**3**	**1**	**4**

Shortest Paths from 0:

0 ⟶ 0 = 0 (trivial)
0 ⟶ 1 = 3 via path: 0 ⟶ 2 ⟶ 1 (1 + 2 = 3, better than direct 4!)
0 ⟶ 2 = 1 via path: 0 ⟶ 2
0 ⟶ 3 = 4 via path: 0 ⟶ 2 ⟶ 1 ⟶ 3 (1 + 2 + 1 = 4)

C++ Code (Min-Heap)

```
void dijkstra(vector<pair<int,int>> adj[], int V, int src) {
    vector<int> dist(V, INT_MAX);
    priority_queue<pair<int,int>,
        vector<pair<int,int>>, greater<>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;  // outdated entry
        for (auto [v, w] : adj[u])
            if (dist[u]+w < dist[v]) {
                dist[v] = dist[u]+w;
                pq.push({dist[v], v});
            }
    }
}
```

16

Kruskal's Algorithm (MST)

Medium

O(E log E)

📖 Definition

Finds **Minimum Spanning Tree**. Sort all edges by weight → Add edges one by one (greedily) → Skip edge if it forms a **cycle** (checked via Union-Find). Stop when V-1 edges added.

📐 MST Properties

MST has exactly V-1 edges

Connects all V vertices

Minimum total weight

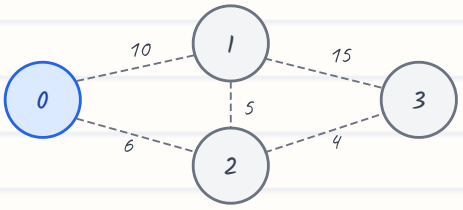
No cycles!

⚡ Complexity

Step	Time
Sort edges	O(E log E)
Union-Find	O(α(V)) ≈ O(1)
Total	O(E log E)
Space	O(V)

🔢 Numerical — Find MST of graph (4 vertices)

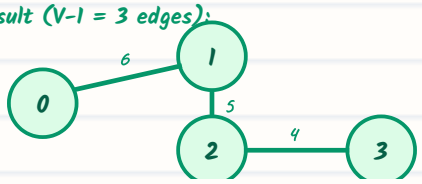
Graph:



Sorted Edges by Weight:

Edge	Weight	Add?	MST Cost
2–3	4	✓ YES	4
1–2	5	✓ YES	9
0–2	6	✓ YES	15
0–1	10	✗ CYCLE	—
1–3	15	✗ CYCLE	—

MST Result (V-1 = 3 edges):



MST Edges:
0–2 (w=6)
1–2 (w=5)
2–3 (w=4)
Total = 15

C++ Code (Union-Find)

```
int find(int parent[], int i) {
    return parent[i]==i ? i : parent[i]=find(parent,parent[i]);
}
void kruskal(vector<tuple<int,int,int>>& edges, int V) {
    sort(edges.begin(), edges.end()); // sort by weight
    vector<int> parent(V);
    iota(parent.begin(), parent.end(), 0);
    int cost = 0, cnt = 0;
    for (auto [w, u, v] : edges) {
        int pu=find(parent.data(),u), pv=find(parent.data(),v);
        if (pu != pv) {   // no cycle!
            parent[pu] = pv;
            cost += w; cnt++;
            if (cnt == V-1) break;
        }
    }
    cout << "MST Cost = " << cost;
}
```

09

Huffman Coding

Hard

O(n log n)

📖 Definition

Lossless data compression using greedy approach. Assigns *shorter binary codes* to more frequent characters. Build a **min-heap** tree bottom-up.

📐 Key Rules

- Left child → 0, Right child → 1
- Always merge 2 nodes with lowest freq
- Result: **Prefix-free codes**
- No code is prefix of another

⚡ Complexity

Step	Time
Build heap	O(n)
Build tree	O(n log n)
Space	O(n)

✓ Lossless ✓ Optimal prefix

Used in: ZIP, JPEG, MP3, PNG

🔢 Numerical — Characters: a(5), b(9), c(12), d(13), e(16), f(45)

Building Huffman Tree (Min-Heap merges):

Step 1: Merge min a(5) + b(9) = node(14)

Step 2: Merge min c(12) + d(13) = node(25)

Step 3: Merge node(14) + e(16) = node(30)

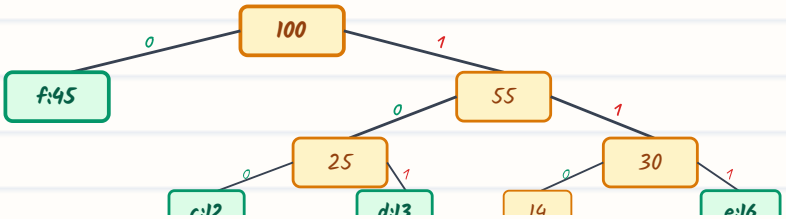
Step 4: Merge node(25) + node(30) = node(55)

Step 5: Merge f(45) + node(55) = ROOT(100)

Huffman Tree:

Huffman Codes

f → 0 (45, 1 bit)
c → 100 (12, 3 bits)
d → 101 (13, 3 bits)
e → 111 (16, 3 bits)
a → 1100 (5, 4 bits)
b → 1101 (9, 4 bits)



📊 Algorithm Complexity Quick Reference

Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable	Notes
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	✅	Simple, educational
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	❌	Min swaps (n-1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	✅	Best for nearly sorted
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	✅	Linked lists, external
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	❌	Fastest in practice
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	❌	Guaranteed O(n log n)

Graph Algorithms

Algorithm	Time	Space	Purpose	Key Limitation
BFS	O(V+E)	O(V)	Shortest path (unweighted)	Queue needed
DFS	O(V+E)	O(V)	Cycle detect, Topo Sort	Not shortest path
Dijkstra	O((V+E)logV)	O(V)	SSSP non-neg weights	No negative edges!
Bellman-Ford	O(VE)	O(V)	SSSP with neg weights	Slower than Dijkstra
Floyd-Warshall	O(V³)	O(V²)	All-pairs shortest path	Cubic time!
Kruskal's	O(E log E)	O(V)	MST — sparse graphs	Union-Find needed
Prim's	O(E log V)	O(V)	MST — dense graphs	Priority queue needed
Topological Sort	O(V+E)	O(V)	Ordering in DAG	DAG only!

Greedy Algorithms

Algorithm	Time	Greedy Choice	Works?
Activity Selection	O(n log n)	Earliest finish time	✅ Always optimal
Fractional Knapsack	O(n log n)	Highest value/weight ratio	✅ Always optimal
Huffman Coding	O(n log n)	Merge 2 lowest freq nodes	✅ Always optimal
Job Sequencing	O(n²)	Highest profit first	✅ Always optimal
Coin Change	O(n)	Largest coin first	⚠️ Canonical systems only
0/1 Knapsack	O(nW)	Greedy FAILS — use DP!	❌ Use DP

🎯 Exam Tips

📌 Negative edges → Bellman-Ford, NOT Dijkstra

📌 All pairs shortest path → Floyd-Warshall $O(V^3)$

📌 MST: Kruskal (sparse), Prim (dense)

📌 Merge Sort: stable, $O(n \log n)$ guaranteed, extra $O(n)$ space

📌 Quick Sort: fastest in practice, $O(n^2)$ worst case

📌 BFS = shortest path unweighted; DFS = cycle detect, topo sort

📌 Heap Sort: guaranteed $O(n \log n)$, in-place, unstable

📌 Insertion Sort: $O(n)$ best for nearly-sorted arrays